

28. Вградени компютри с намалена консумация и повишена надеждност.

Алгоритми за откриване и коригиране на грешки при комуникация.

1. Вградени компютри с намалена консумация (Low-Power Embedded Systems)

В съвременните вградени системи (като IoT у-ва, медицински у-ва, космическа апаратура, сензорни мрежи) управлението на енергията е критичен фактор. Намалената консумация удължава живота на батерията и намалява топлинното отделяне, което директно рефлектира върху надеждността на компонентите.

1.1 Източници на консумация в CMOS схеми

Общата консумирана мощност (P_{total}) в една цифрова интегрална схема (напр. микроконтролер) се дефинира като сума от динамичната и статичната мощност:

$$P_{total} = P_{dynamic} + P_{static}$$

- Динамична мощност ($P_{dynamic}$): Консумира се по време на превключване на логическите нива (зареждане и разреждане на паразитните капацитети) и от токовете на късо съединение по време на превключването.

$$P_{dynamic} = \alpha \cdot C \cdot V_{dd}^2 \cdot f$$

- Където: α е коефициент на активност, C е капацитет на товара, V_{dd} е захранващото напрежение, а f е работната честота.
- Статична мощност (P_{static}): Консумира се, когато схемата е в покой (няма превключване). Дължи се главно на токовете на утечка (leakage currents) през транзисторите.

$$P_{static} = I_{leak} \cdot V_{dd}$$

1.2 Хардуерни методи за намаляване на консумацията

- Динамично мащабиране на напрежението и честотата (DVFS - Dynamic Voltage and Frequency Scaling):** Тъй като $P_{dynamic}$ зависи квадратично от напрежението и линейно от честотата, намаляването им при ниско натоварване на процесора води до огромни спестявания на енергия.
- Управление на тактовия сигнал (Clock Gating):** Изключване на тактовия сигнал към периферия или блокове от процесора, които не се използват в момента. Това спира превключването на транзисторите и свежда динамичната консумация в този блок до нула.
- Управление на захранването (Power Gating):** Пълно изключване на захранващото напрежение към определени хардуерни домейни. Премахва както динамичната, така и статичната мощност (токовете на утечка).

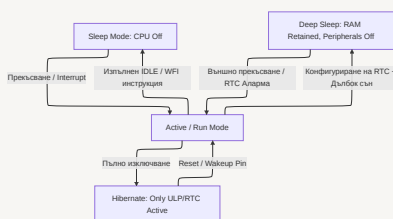
1.3 Софтуерни методи за намаляване на консумацията

Съвременните микроконтролери (напр. ARM Cortex-M, ESP32) предлагат архитектура с няколко енергийни домейна и режими на сън.

Режим (Mode)	Активни компоненти	Консумация (Типична)	Време за събуждане
Active / Run	CPU, Периферия, Flash, RAM	10 – 50 mA	0 ns

Sleep / Idle	Периферия, RAM, CPU тактов сигнал е спрян	1 – 5 mA	Няколко такта
Deep Sleep	Основно CPU е изключено, RTC работи, запазена RAM	10 – 50 μ A	Милисекунди
Hibernate / Standby	Изключено всичко без ULP (Ultra-Low-Power) копроцесор или таймер	< 5 μ A	Най-дълго (студен рестарт)

Пример: Безжичен температурен сензор измерва данни на всеки 10 минути. Вместо да работи постоянно в Active Mode (20 mA), софтуерът го приспива в Deep Sleep (15 μ A). Устройството се събужда чрез хардуерен таймер (RTC) за 200 милисекунди, изпраща данните през Wi-Fi/BLE и отново заспива. Това удължава живота на батерията от 2 дни на 3 години.



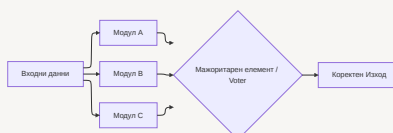
2. Повишена надеждност

Надеждността е свойството на системата да изпълнява своята дефинирана функция при определени условия за даден период от време. Във вградените компютри грешките се делят на:

1. **Постоянни:** Физическа повреда на хардуера.
2. **Времени:** Предизвикани от външни смущения, които променят състоянието на паметта, без да повреждат физически чипа.

2.1 Хардуерна излишност

За системи с критично значение (авиация, автомобилни спирачни системи) се използва Тройна модулна излишност (TMR - Triple Modular Redundancy).



Три идентични хардуерни модула изпълняват една и съща задача едновременно. Резултатите им се подават на мажоритарен логически елемент (Voter). Ако в един от модулите възникне грешка (напр. Модул А върне 0, а Модул В и С върнат 1), гласуващият елемент избира 1 и маскира грешката.

2.2 Времева и софтуерна излишност

- **Времева излишност:** Повторно изпълнение на една и съща програма или изчисление при откриване на несъответствие. Ефективно при временни грешки.
- **Watchdog Timer:** Независим хардуерен таймер, който брой обратно до нула. Софтуерът трябва периодично да го "нулира". Ако софтуерът забие в безкраен цикъл или блокира поради смущение, таймерът достига нула и генерира хардуерен рестарт (Reset) на системата.

2.3 Brown-ови детектори (Brown-out Reset - BOR)

Когато захранващото напрежение падне под критичен минимум (напр. при разреждане на батерия или пикова консумация), логическите нива в микроконтролера стават недефинирани, което води до корупция на данни и

непредвидимо поведение. BOR схемата следи напрежението и държи процесора в състояние на рестарт, докато захранването не се стабилизира.

3. Алгоритми за откриване и коригиране на грешки при комуникация

При предаване на данни по комуникационни канали (като UART, I2C, CAN, SPI) шумовете могат да променят битове ($0 \rightarrow 1$ или $1 \rightarrow 0$). За осигуряване на надеждност се използват алгоритми за откриване на грешки и коригиране на грешки.

3.1 Алгоритми за откриване на грешки

3.1.1 Бит за четност

Най-простият метод. Към данните (напр. 8 бита) се добавя 1 допълнителен бит, така че общият брой на единиците в пакета да бъде четен (Even parity) или нечетен (Odd parity).

Пример (Even Parity):

- Данни за изпращане: **1011001** (4 единици $\rightarrow$ четен брой). Парити битът става 0. Пакет: **10110010**.
- Ако възникне 1-битова грешка по канала: 10110110 (5 единици). Получателят открива, че броят е нечетен и отхвърля пакета.

Ограничение: Не може да открие двойни грешки (две промени взаимно се компенсират).

3.1.2 Контролна сума (Checksum)

Всички байтове от съобщението се събират аритметично. Резултатът (често в допълнение до 1 или 2) се прикачва накрая на съобщението. Получателят извършва същото събиране и сравнява резултата. Използва се масово в мрежовите протоколи (като IP, TCP, UDP).

3.1.3 Цикличен излишен код

CRC е мощен подход, базиран на полиномно делене в полето на Галоа ($GF(2)$), което означава, че събирането и изваждането са еквивалентни на операцията XOR.

Данните се разглеждат като коефициенти на полином $M(x)$, който се дели на предварително дефиниран генериращ полином $G(x)$. Остатъкът от това деление е CRC сумата.

Кратък пример за CRC изчисление:

Данни (M): **101100**

Генериращ полином (G): 1011 (съответства на $x^3 + x + 1$, степен $n = 3$)

- Стъпка 1: Добавят се n нули в края на данните: **101100000**
- Стъпка 2: Извършва се XOR деление:

```
101100000 / 1011
1011
----
000000000 -> Остатъкът е 000.
```

(За простота в този пример данните се делят точно, така че CRC = 000. Ако имаше остатък, например 110, той щеше да се добави към съобщението).

CRC е изключително ефективен за откриване на пакетни грешки (burst errors) – ситуации, в които няколко последователни бита са компрометирани от краткотраен силен шум.

3.2 Алгоритми за коригиране на грешки (Error Correction)

Коригирането на грешки изисква въвеждането на по-голяма информационна излишност. Ключово понятие тук е Разстояние на Хаминг (Hamming Distance) - броят на позициите, в които два кодови низа с еднаква дължина се различават.

- За откриване на e на брой грешки е необходимо минимално разстояние $d_{min} \geq e + 1$.
- За коригиране на t на брой грешки е необходимо минимално разстояние $d_{min} \geq 2t + 1$.

Код на Хаминг (Hamming Code)

Класически алгоритъм за коригиране на единични грешки (SEC - Single Error Correction) и откриване на двойни грешки (DED - Double Error Correction).

При Код на Хаминг контролните битове (parity bits) се позиционират на места, които са степени на двойката (1, 2, 4, 8, 16...). Всеки контролен бит отговаря за четността на специфична комбинация от информационни битове.

Пример за Хаминг Код (7, 4)

Кодират се 4 бита данни с 3 контролни бита, обща дължина 7 бита.

Нека данните са $D = [d_3, d_2, d_1, d_0] = [1, 0, 1, 1]$.

Разположение на битовете в 7-битовата дума (P - parity, D - data):

Позиция	1	2	3	4	5	6	7
Тип бит	P1	P2	D3	P4	D2	D1	D0
Стойност	0	1	1	0	0	1	1

Формули за определяне на контролните битове (чрез XOR / Even parity):

- P_1 проверява позициите, чийто бинарен индекс завършва на 1 (1, 3, 5, 7):

$$P_1 = D_3 \oplus D_2 \oplus D_0 = 1 \oplus 0 \oplus 1 = 0$$

- P_2 проверява позициите, чийто бинарен индекс има 1 на втората позиция (2, 3, 6, 7):

$$P_2 = D_3 \oplus D_1 \oplus D_0 = 1 \oplus 1 \oplus 1 = 1$$

- P_4 проверява позициите, чийто бинарен индекс има 1 на третата позиция (4, 5, 6, 7):

$$P_4 = D_2 \oplus D_1 \oplus D_0 = 0 \oplus 1 \oplus 1 = 0$$

Изпратената дума е: **0110011**.

Декодиране и коригиране на грешка

Ако по време на комуникацията позиция 6 се промени от 1 на 0 поради шум, получателят получава низ: **0110001**.

Получателят изчислява т.нар. синдром (Syndrome) чрез повторно изчисляване на паритетите, включвайки самите контролни битове:

- $S_1 = P_1 \oplus D_3 \oplus D_2 \oplus D_0 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$
- $S_2 = P_2 \oplus D_3 \oplus D_1 \oplus D_0 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$
- $S_4 = P_4 \oplus D_2 \oplus D_1 \oplus D_0 = 0 \oplus 0 \oplus 0 \oplus 1 = 1$

Подреждаме синдрома отзад напред: $S = [S_4, S_2, S_1] = [1, 1, 0]$.

Бинарната стойност 110 е равна на 6 в десетична бройна система. Това директно показва, че грешката е в позиция 6. Получателят просто инвертира бита на позиция 6 ($0 \rightarrow 1$) и успешно възстановява оригиналното съобщение без нужда от препредаване.

Алгоритъм	Откриване на грешки	Коригиране на грешки	Излишност (Overhead)	Приложение
Parity	Само единични грешки	Не	Много ниска (1 бит)	UART, Прости терминали

Checksum	Средно ниво (блокови)	Не	Ниска (8/16 бита)	Мрежови стекове (TCP/IP)
CRC	Отлично (пакетни грешки)	Не	Средна (16/32 бита)	Ethernet, CAN, SPI, Flash памети
Hamming	Единични и двойни	Единични грешки	Висока (при (7,4) е ~43%)	ECC RAM памети, Сателитна комуникация